

Prototype Augmentation-based Edge-end Heterogeneous Collaborative Learning

Enze Yu
Nanjing University
Nanjing, China
enzeyu@smail.nju.edu.cn

PengHuan Cheng
Nanjing University
Nanjing, China
cph@smail.nju.edu.cn

Haipeng Dai
Nanjing University
Nanjing, China
haipengdai@nju.edu.cn

Haihan Zhang
Nanjing University
Nanjing, China
zhzh@smail.nju.edu.cn

Sujin Hou
Jilin University
Changchun, China
housj23@mails.jlu.edu.cn

Meng Li
Nanjing University
Nanjing, China
meng@nju.edu.cn

Zhenzhe Zheng
Shanghai Jiao Tong University
Shanghai, China
zhengzhenzhe@sjtu.edu.cn

Qiang He
Huazhong University of Science and
Technology
Wuhan, China
hqliang@hust.edu.cn

Guihai Chen
Nanjing University
Nanjing, China
gchen@nju.edu.cn

Abstract

Collaborative learning between edge servers (e.g., base stations) and end devices (e.g., drones) enables simultaneous model training in web applications through knowledge sharing. The resulting models effectively reduce service latency. However, existing approaches either assume isomorphic models on edge servers and end devices or incur substantial transmission overhead when training. Moreover, edge servers are often unable to access data from end devices on time due to long-distance constraints or strict data privacy regulations. This paper proposes a **Prototype Augmentation-based Edge-end Collaborative Learning** method (PAECL). *It simultaneously trains heterogeneous edge and end models in the absence of data on edge servers by transmitting only augmented class-wise feature vectors (prototypes), significantly reducing communication overhead compared to sharing models, data, or logits.* Specifically, on end devices, prototype-implied latent knowledge is augmented via local prototype contrast and global prototype alignment. On edge servers, prototypes are further augmented to produce bounded virtual vectors by mixing them with random noise, and the augmented prototypes are then delivered to generative models to provide data during edge model training. Through simulations and field experiments, PAECL achieves the highest accuracy for edge and end models under limited training resources and reduces the transmission burden by at least 297 times compared to existing edge-end heterogeneous learning methods.

CCS Concepts

• **Human-centered computing** → Ubiquitous and mobile computing; • **Computing methodologies** → Machine learning.



This work is licensed under a Creative Commons Attribution 4.0 International License. WWW '26, Dubai, United Arab Emirates.
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792281>

Keywords

Edge-end collaborative learning; Prototype augmentation; Heterogeneous models

ACM Reference Format:

Enze Yu, PengHuan Cheng, Haipeng Dai, Haihan Zhang, Sujin Hou, Meng Li, Zhenzhe Zheng, Qiang He, and Guihai Chen. 2026. Prototype Augmentation-based Edge-end Heterogeneous Collaborative Learning. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3774904.3792281>

1 Introduction

Fueled by advances in Artificial Intelligence (AI) and on-device computing, collaborative model training across ubiquitous devices has gained significant attention [7, 34, 43]. Edge-end collaborative learning implements it by sharing knowledge (e.g., data and models) between servers and end devices [9, 23, 38]. It yields models that are closer to users, reducing latency and enabling real-time applications such as video analysis [14], recommender systems in web applications [17], and smart transportation [2]. For instance, in web applications, edge-end collaboration can be leveraged to train recommendation models by jointly utilizing user interactions on end devices and computationally intensive models on edge servers [22]. End devices locally extract and upload personalized features from user behaviors (e.g., browsing and click history). The edge servers aggregate these representations and train larger-scale recommendation models.

Most prior studies concentrate on homogeneous model settings, where servers usually only perform knowledge aggregation, resulting in insufficient utilization of server resources [18, 24, 47]. In contrast, we focus on edge-end heterogeneous collaborative learning (EHCL), where servers train large-scale models and end devices train small-scale models. It brings three advantages: (1) During training, small models can learn from the output of the large model to improve their performance [30]. (2) It enables

comprehensive resource utilization of servers [33], extending their role beyond model aggregation and transmission. (3) After training, the small model can send uncertain samples to the large model for further inference [34].

Unfortunately, implementing EHCL is difficult because of different model architectures, limited communication resources, and uneven data distribution. Existing studies commonly adopt federated learning frameworks to implement EHCL [35, 40, 46]. However, these methods primarily focus on heterogeneous end-side models [32, 50] or train homogeneous models [3, 37, 45], resulting in underutilized resources of edge servers. The authors in [9, 25, 38] train different models by sharing logits or encoded data, but their transmission burden scales with data volume, rendering them impractical for edge-end networks. Typically, end devices have more data but limited resources, whereas edge servers have more resources but less data. Edge servers also cannot obtain device data on time due to legal and distance restrictions [1, 8]. This uneven data distribution [42] and communication resources gap [39] further hinders training efficiency.

Prototypes, as the average feature vectors of data within the same class, have merely 5% communication cost of other gradient-based methods. Many researchers have employed it to train heterogeneous end models [11, 25, 27, 31, 49, 50]. However, they do not consider heterogeneous edge model training, and the transmission burden is huge due to the hybrid transmission of logits and prototypes. They also cannot be used for edge model training due to data lack. *To address these problems, we explore its potential for edge-end heterogeneous learning by using prototypes as the sole knowledge-sharing carrier.*

Nonetheless, implementing the above ideas incurs the following challenges. **Challenge 1:** Although prototypes are a powerful knowledge carrier, simply forcing prototypes in end devices to be close to aggregated prototypes impairs the preservation of local discriminative information. Additionally, edge servers cannot train models with aggregated prototypes due to inconsistencies in edge and end models. **Challenge 2:** The uneven data distribution and computing resources gap further complicate heterogeneous edge-end model training because end devices are usually unable to transmit data directly due to privacy regulations and limited computing resources.

For **Challenge 1**, we perform prototype contrast and alignment on end devices, which enables prototypes to obtain more local information. We innovatively augment prototypes on the edge servers by mixing noise with the global prototype. Then, the augmented prototypes are exploited to train heterogeneous edge models. For **Challenge 2**, edge servers aggregate prototypes and generate data on the server. This training paradigm utilizes the resources of edge servers. Our main contributions can be summarized as follows.

- To the best of our knowledge, this paper is the first work that solves challenges of heterogeneous edge-end models training and data lacking on edge servers by collaboratively transmitting only prototypes, which are extremely lightweight and complementary to existing knowledge-sharing methods.
- We propose a novel method, PAECL, which trains heterogeneous edge-end models by augmenting prototypes on edge

servers and end devices. Specifically, end devices first supplement knowledge and augment prototypes through local prototype contrast and global prototype alignment. Edge servers amplify class-specific information from prototypes by boundedly mixing prototypes with noise, which is further utilized by generative models to produce data.

- We conduct extensive evaluations on four datasets and three model architectures through both simulations and field experiments. Results demonstrate that PAECL achieves superior accuracy for both edge and end models in varying network scenarios. It reduces the transmission burden by at least 297 times, and edge model accuracy on four datasets is at least 5.61%, 19.51%, 7.86%, and 6.80% higher than existing edge-end heterogeneous collaborative training methods.

2 Related Work

2.1 Edge-end Collaborative Learning

Edge-end collaborative learning methods are divided into homogeneous and heterogeneous training by different objectives. **Homogeneous models:** Current edge-end collaborative training primarily focuses on homogeneous models. The mainstream federated learning methods in edge networks [36, 41, 44] can be directly migrated to homogeneous model training, and they rely on models, logits, or prototypes to transmit knowledge among participants. In [47], data heterogeneity is alleviated by adjusting local model aggregation weights. The authors in [24] proposed to obtain the global model by aggregating all local models, then used it for knowledge distillation. Split learning [18, 19] is used to transfer logits between client and server for federated learning. Recently, prototypes have greatly reduced the communication burden as the knowledge carrier for federated learning [31]. **Heterogeneous models:** Existing heterogeneous federated learning methods [15, 45] mainly focus on training personalized models on the end side, which cannot meet the requirements of EHCL. For edge-end heterogeneous model training, researchers use encoded data and logits for training. The authors in [38] proposed to utilize a lightweight encoder-decoder to process data and generate bridge samples. These bridge samples are sent to edge servers to train a larger-scale model. Logits is also transferred between edge servers and end devices to complete larger edge model training [9].

The above homogeneous methods merely treat the server as an aggregator, underutilizing its computational resources. Differently, we consider a realistic scenario of multiple edge servers and end devices, enabling them to train heterogeneous models by transmitting lightweight prototypes. In this way, the transmission burden increases with the number of classes rather than data volumes, which is more friendly to end devices.

2.2 Prototype-based Learning

A prototype is defined as the mean feature embedding vectors of samples belonging to the same class. In the early stages, it is used for transfer learning [6, 26, 28] and few-shot learning [5, 20, 29]. Current prototype work primarily focuses on generating personalized end-side models [4, 11, 31, 48]. For example, multiple drones collaboratively conducted prototype-based joint optimization and

R-1 Prototype	R-10 Prototype	R-100 Prototype
49.81%	53.04%	57.60%

Table 1: Edge model accuracy with different prototypes.

training to achieve good training performance by selecting prototypes [48]. The authors in [10, 11] constructed unbiased prototypes by clustering local prototypes, which addresses the domain shift problem in end devices. Class prototypes are further abstracted to mitigate representation bias and enhance the generalization ability of the learned models [4, 50]. The mixed transmission of prototypes and logits to improve the performance of heterogeneous models was also explored [25]. However, the above work either trains heterogeneous end models or causes a huge transmission burden. In contrast, we use prototypes as the sole carrier to conduct EHCL from a new perspective. On the one hand, we only transfer prototypes in edge-end and end-end transmission, ensuring a low transmission burden. On the other hand, we train edge models by producing data with prototype augmentation.

3 Problem and Motivation

3.1 Problem Statement

In an EHCL scenario, M edge servers and N end devices collaborate to train models with different architectures. We let $\mathbf{M}$ and $\mathbf{N}$ denote the set of edge servers and end devices. W_d and W_e are two different model architectures deployed on the end devices and edge servers. End device n owns private data D_n and a predefined model w_n . D_N denotes the total number of samples over all end devices. In contrast, edge server m with relatively sufficient computing resources usually lacks training data and maintains a model w_m with larger capacity. w_n and w_m are instantiation models for W_d and W_e . The number of classes of all end devices in the network is C .

Formally, for each end device, its objective is shown in Eq. (1), where $|D_n|$ is the sample number of D_n and $\mathcal{L}_n(w_n; D_n)$ is the local loss function.

$$\arg \min_{w_1, \dots, w_N} \sum_{n=1}^N \frac{|D_n|}{D_N} \mathcal{L}_n(w_n; D_n). \quad (1)$$

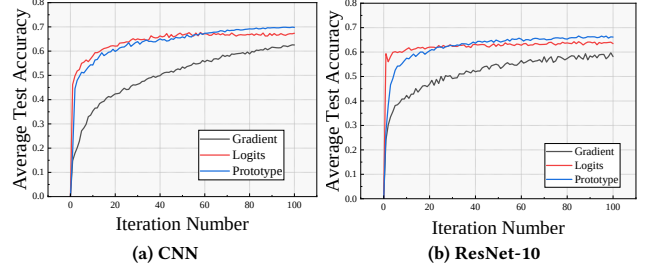
The optimization objective of each edge server is formulated in Eq. (2), where $\mathcal{L}_m(w_m; D_m)$ is the loss function of the edge server m . In practice, D_m is often empty, as edge servers cannot directly obtain or access data from end devices in a timely manner due to long distances and user privacy constraints. In such scenarios, it is difficult for edge models to perform well with a small test dataset.

$$\arg \min_{\omega_1, \dots, \omega_M} \sum_{m=1}^M \frac{1}{M} \mathcal{L}_m(w_m; D_m). \quad (2)$$

3.2 Motivation of Augmenting Prototypes

Traditional knowledge-sharing means of collaborative learning mainly consist of gradient [21], model [3] and logits [9]. As a lightweight knowledge-sharing method, prototypes have been used to improve end models' performance in a data heterogeneity scenario [31]. As a necessary communication component, we ask the question: *Does the aggregated prototype have the potential to improve the performance of heterogeneous edge models?*

Previous prototype-based schemes have not explored the above problem and only augment prototypes from the perspective of end


Figure 1: Test accuracy with three knowledge-sharing ways.

models. Actually, prototypes are essentially feature vectors, which contain class information but cannot be used directly for training. To this end, we are inspired by the class centrality property of the prototype and augment the capacity of prototypes for EHCL. We augment prototypes from a new perspective of edge servers. Specifically, we consider using prototypes as the full-dimensional vector in a multi-dimensional space, generating a small number of virtual feature vectors by mixing the prototype and random noise. Virtual feature vectors are given to the pretrained generative models, and the generated data is utilized to train on edge servers.

We conduct a preliminary experiment to verify the idea, utilizing three global prototypes generated by a 2-layer CNN at different communication rounds: 1, 10, and 100. Gaussian noises are randomly generated and added to these prototype vectors. Then, different vectors are passed to the pre-trained generative model StyleGAN [12], producing multiple generated samples. We set the edge model architecture as a 6-layer CNN, and generate 1000 samples for each class. All generated samples are used for training, and the test dataset of CIFAR-10 is used to test. In Table 1, R-1 Prototype, R-10 Prototype, and R-100 Prototype represent the prototype vectors of the 1th, 10th, and 100th rounds, respectively. As shown in Fig. 1, it can be seen that the Prototype method achieves the highest test accuracy among other two methods. We can conclude that generated samples from prototypes improve the edge model's performance. As the iteration rounds increase, the global prototype in 100th round can bring the greatest performance improvements.

Therefore, edge-end heterogeneous model performances can be improved simultaneously when only prototypes are transmitted. This motivates us to propose PAECL.

4 PAECL

4.1 Overview

As shown in Fig. 2, edge servers and end devices conduct collaborative training by the transmission of augmented prototypes. The aggregation edge server sends global prototype to other servers. In terms of end devices, by contrasting and aligning, end prototypes are augmented and refined with the help of global knowledge. Besides, edge servers augment the global prototype and expand the class feature vector into a high-dimensional space to enrich the feature representation. Then, the pre-trained generative models are used to generate images for training edge models. Therefore, our proposed PAECL effectively conducts EHCL in a typical network scenario by continuously augmenting prototypes at edge servers and end devices.

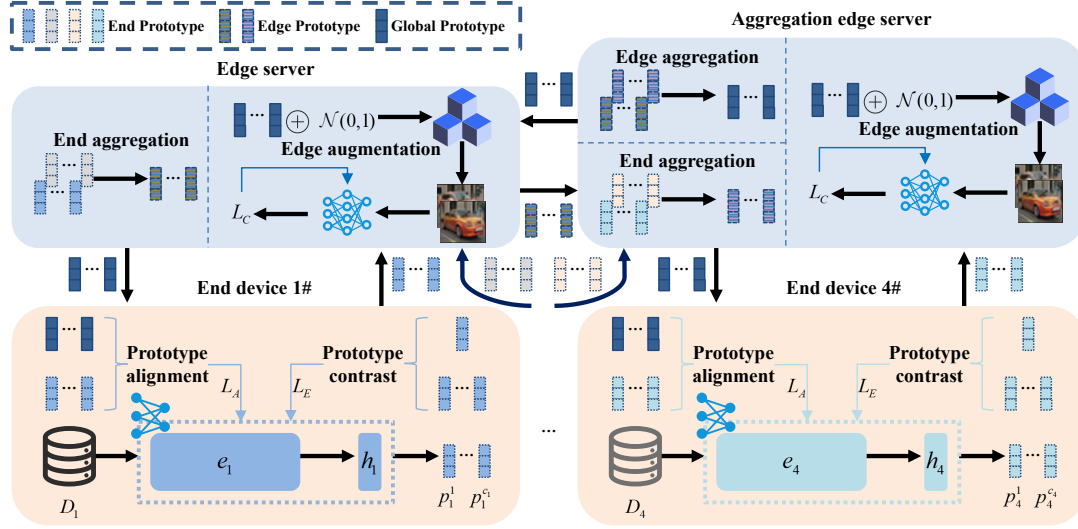


Figure 2: A toy example of PAECL with two edge servers and four end devices. End devices 2# and 3# are omitted; only their end prototypes are provided. Three different borders are chosen to distinguish end, edge, and global prototypes. Different colors represent prototypes generated from different devices. Ellipses represent different class prototypes.

4.2 End, Edge, and Global prototype

We first define the end, edge, and global prototypes. They are carriers of the knowledge transmission of PAECL and are augmented.

End network architecture. Generally, every end model can be seen as a combination of two components. Take client n and its model w_n as an example; its components are a feature encoder and a projection network. (1) Feature encoder $e(f_n; \cdot): \mathbb{R}^d \rightarrow \mathbb{R}^{d_e}$, it map the data sample with size d to a feature representation of size d_e . (2) Projection network $h(\theta_n; \cdot): \mathbb{R}^{d_e} \rightarrow \mathbb{R}^{d_h}$, and it maps $e(f_n; \cdot)$ from a feature space with size d_e to another space with size d_h for further learning or classification.

End prototype. First, we extract class-relevant knowledge from every end device, and end prototypes are constructed as the knowledge carrier to share. It is represented in the latent space of the projection network's output and described as the average of all the feature representations within the same class.

$$\bar{p}_n^j = \frac{1}{|D_n^j|} \sum_{(x,y) \in D_n^j} h_n(\theta_n; e_n(f_n; x)). \quad (3)$$

In Eq. (3), we define the end prototype $\bar{p}_n^j$ of class j belonging to device n , where D_n^j is the subset of D_n composed of all samples belonging to class j . If edge device n has c_n categories, then its corresponding end prototype is a set $\bar{p}_n = \{\bar{p}_n^1, \dots, \bar{p}_n^{c_n}\}$, where each element is a feature vector. After each end device completes the calculation of the end prototype, the end prototypes of different devices will be sent to the corresponding edge server for prototype aggregation to generate edge prototypes.

Edge and global prototype. All edge servers calculate their edge prototype upon receiving end prototypes from devices they manage. Taking server m as an example, its edge prototype $\hat{p}_m^j$ of class j is computed in Eq. (4), where S_m^j denotes the subset that end devices have samples of class j . The subset D_m^j represents the set of

samples belonging to class j in end device n and D_m^j represents the set of all samples belonging to class j in all end devices managed by the edge server m . The prototype of edge server m is also a set, denoted by $\hat{p}_m = \{\hat{p}_m^1, \dots, \hat{p}_m^{c_m}\}$, where c_m is the total number of class of end devices managed by the edge server m .

$$\hat{p}_m^j = \sum_{n \in S_m^j} \frac{|D_n^j|}{|D_m^j|} \bar{p}_n^j. \quad (4)$$

An edge server is chosen as the aggregation edge server. It aggregates edge prototypes and yields the global prototype, which embodies the concentrated knowledge and is a highly condensed representation. The aggregation edge server computes the global prototype P_g^j of class j as

$$P_g^j = \sum_{m \in M} \frac{|D_m^j|}{|D_M^j|} \hat{p}_m^j, \quad (5)$$

where D_M^j is the subset of samples of class j owned by all devices in the network. The global prototype $P_g = \{P_g^1, \dots, P_g^{C_g}\}$, indicates that each class has a corresponding vector. The aggregation edge server will send the global prototype to all edge servers, and they will forward it to all corresponding end devices.

4.3 Augmenting Prototypes in End Devices

As shown in function **endTraining()** of Alg. 1, end devices supply knowledge and augment prototypes by contrast and alignment. In practical scenarios, end devices have missing classes. For end device n , it supplies its knowledge as

$$\bar{p}_n^j = \begin{cases} \bar{p}_n^j, & j \in C_n \\ P_g^j, & j \notin C_n \end{cases} \quad (6)$$

Algorithm 1 PAECL. Edge serves and end devices train their heterogeneous models by augmenting prototypes.

Input: Datasets and models of end devices $\{D_n\}_{n=1}^N, \{w_n\}_{n=1}^N$;
Models of edge servers $\{w_m\}_{m=1}^M$.

Output: $\{w_n\}_{n=1}^N, \{w_m\}_{m=1}^M$.

Every edge server m execute

```

1: for global communication round  $t=1, \dots, T$  do
2:    $\{\tilde{p}_n\}_{n=1}^{S_m} \leftarrow \text{endTraining}()$  called by every end device covered by the current edge server.
3:   Receive  $\{p_n\}_{n=1}^{S_m}$ , compute edge prototype  $\hat{p}$  by Eq. (4), and send the edge prototype.
4:   Receive and distribute global prototype  $P_g$ .
5:   Augment the global prototype  $P_g$  by Eq. (11).
6:   Generate auxiliary dataset  $D_{aux}$  by Eq. (12) and use it to train its edge model.
7:   if Edge server  $m$  is aggregation edge server then
8:     Compute the global prototype  $P_g$  by Eq. (5), then distribute to other edge servers.
9:   end if
10: end for
11: Obtaining trained models  $\{w_n\}_{n=1}^N$  and  $\{w_m\}_{m=1}^M$ .
```

Function endTraining():

```

1: Supplement the knowledge by Eq. (6).
2: for each batch in  $D_n$  do
3:   Compute  $\mathcal{L}_E$  by Eq. (7).
4:   Compute  $\mathcal{L}_A$  by Eq. (8).
5:   Train the end model by Eq. (9).
6: end for
7: Compute end prototype  $\bar{p}$  by Eq. (3).
8: return  $\bar{p}$ .
```

where C_n is the class set of end device n before receiving the global prototype. Different from Eq. (3), Eq. (6) makes end devices obtain knowledge they lack through transmitting the global prototype.

After supplementing knowledge, end devices conduct local training with the received global prototype P_g . Local training aims to extract as much knowledge as possible from P_g with the help of a local dataset. Therefore, we propose a local prototype contrast loss $\mathcal{L}_E$ and a global prototype alignment loss $\mathcal{L}_A$.

Local prototype contrast: For an end device n , given a labeled sample (x, y) , its output produced by the projection network is p_x . The prototype of class y is $\bar{p}_n^y$ and prototypes of other classes are indexed by y' . we define $\mathcal{L}_E$ as a contrast loss in Eq. (7), where cosine similarity $\cos(p_x, \bar{p}_n^y) = \frac{p_x^T \bar{p}_n^y}{\|p_x\|_2 \cdot \|\bar{p}_n^y\|_2}$ measures the distance between p_x and $\bar{p}_n^y$. It uses the idea of contrastive learning [13] to force sample x to be close to the local prototype $\bar{p}_n^y$ of the corresponding class and away from the prototypes $\bar{p}_n^{y'}$ of other irrelevant classes.

$$\mathcal{L}_E = -\log \frac{\exp(\cos(p_x, \bar{p}_n^y))}{\exp(\cos(p_x, \bar{p}_n^y)) + \sum_{y'} \exp(\cos(p_x, \bar{p}_n^{y'}))}. \quad (7)$$

Global prototype alignment: The end device n uses the global prototype P_g for training, which is reflected in the alignment loss $\mathcal{L}_A$. It directly supplements knowledge by obtaining global knowledge for missing classes. For the owned classes, the end device iteratively

trains its class prototypes to shorten the distance to the global class prototype. Prototype alignment ensures that the end device trains from a global perspective and acquires knowledge from end devices in other edge servers.

$$\mathcal{L}_A = \sum_{j=1}^{c_n} \|\bar{p}_n^j - P_g^j\|_2. \quad (8)$$

In this way, the local prototype is continuously augmented by reducing contrast and alignment loss. Eq. (9) shows the local training loss function, where $\mathcal{L}_C$ is the cross entropy loss obtained with local data. The loss control parameters w_1 and w_2 decide contrast and alignment loss.

$$L = \mathcal{L}_C + w_1 \mathcal{L}_E + w_2 \mathcal{L}_A. \quad (9)$$

4.4 Augmenting Prototypes in Edge Servers

The global prototype is the sole way to acquire external knowledge for end devices. Therefore, we hope that its transmission can also improve the performance of edge heterogeneous models. The whole workflow of augmenting prototypes in edge servers is shown in Lines 3-8 of Alg. 1. It first aggregates end prototypes and receives the global prototype. Edge servers augment the global prototype by perturbing it to reach other vectors and use augmented results to generate samples for edge model training. We define δ_j as an upper limit of the perturbation radius in Eq. (10).

$$\delta_j = \frac{1}{2} \min_{j \neq j'} \|P_g^j - P_g^{j'}\|_2. \quad (10)$$

Every edge server m has a pre-trained generative model [12]. For the aggregated prototype P_g^j of class j , we generate random vectors p_{ran}^j with the same dimension as the global prototype by sampling from a random distribution $\mathcal{N}(0, \delta_j)$. In Eq.(11), we show a random high-dimensional feature representation obtained by adding the generated random vector corresponding to the global prototype of class j . The generated virtual vector set of edge server is $\bar{p}_{ran}^j = \{\bar{p}_{ran}^{1,j} \dots \bar{p}_{ran}^{n_e,j}\}$, where n_e is the predefined number of generated samples. It aims to limit the virtual vectors generated by each class to remain near its central prototype. Then these random vectors p_{ran}^j are mixed with P_g^j to produce random feature vectors that scatter around the central class prototype in high-dimensional space. This operation is not only easy to implement but also avoids dependence on external datasets.

$$\bar{p}_{ran}^{i,j} = p_{ran}^{i,j} + P_g^j, \quad (11)$$

$$s.t. \|\bar{p}_{ran}^{i,j}\|_2 \leq \delta_j, \quad (11a)$$

In this way, mixing random vectors with the global prototype augments its representation capability, which makes the sole global prototype upgrade to many high-dimensional feature virtual vectors. The edge server m input the virtual vector set $\bar{p}_{ran} = \{\bar{p}_{ran}^1 \dots \bar{p}_{ran}^C\}$ and class set $C = \{1 \dots C\}$ to the generative model G . It generates training samples as an auxiliary dataset as follows:

$$D_{m,aux} = G(\bar{p}_{ran}, C). \quad (12)$$

All the edge servers use the auxiliary dataset to train edge models, and their loss function is the cross-entropy loss. The global prototype is augmented on the edge server, and the auxiliary dataset it generates can improve the performance of edge models.

5 Convergence Analysis of PAECL

We conduct a theoretical analysis of the convergence bound for PAECL. The training process of edge servers is independent due to the generated dataset by augmenting prototypes. Therefore, we focus on the convergence analysis of the training process for end devices. We first give some common assumptions holding for convergence analysis [16].

Assumption 1: All objective functions on end devices are L_1 -smooth, i.e., given any two communication round t_1, t_2 , $\|\nabla \mathcal{L}_{t_1} - \nabla \mathcal{L}_{t_2}\|_2 \leq L_1 \|\omega_{t_1} - \omega_{t_2}\|_2$.

Assumption 2: Stochastic gradient is an unbiased estimator of the local gradient for each end device, i.e., $\forall i \in \{1, 2, \dots, N\}$, $\mathbb{E}_{\xi_i \sim D_i} [g_{i,t_1}] = \nabla \mathcal{L}(\omega_{i,t_1})$.

Assumption 3: The expectation of the stochastic gradient for end devices is bounded by G , i.e., $\forall i \in \{1, 2, \dots, N\}$, $\mathbb{E}[\|g_{i,t_1}\|_2] < G$.

Assumption 4: The feature extraction function for all the end devices is L_2 -smooth, i.e., given any two communication round t_1, t_2 , $\forall i \in \{1, 2, \dots, N\}$, $\|e_i(f_{i,t_1}) - e_i(f_{i,t_2})\| \leq L_2 \|f_{i,t_1} - f_{i,t_2}\|_2$.

We then give the bound for loss function in **Lemma 1**.

Lemma 1: Let **Assumption 1 and 2** hold. From the beginning of the communication round to $\bar{E}$ local update step, the loss function of an end device can be bounded as

$$\mathbb{E}[\mathcal{L}_{(t+1)\bar{E}}] \leq \mathbb{E}[\mathcal{L}_{t\bar{E}}] - \eta \sum_{\bar{e}=0}^{\bar{E}-1} \|\nabla \mathcal{L}_{t\bar{E}+\bar{e}}\|_2^2 + \frac{L_1 \eta^2 \bar{E} G^2}{2}. \quad (13)$$

Proof is shown in appendix. Next, we prove the bound of the loss function after training.

Lemma 2: Let **Assumption 3 and 4** hold. After the end of $(t+1)\bar{E}$ training, we mark the stage at which the global prototype is sent to all end devices with $(t+1)\bar{E} + \frac{1}{2}$. We take the end device i as an example, and its loss function can be bounded as

$$\mathbb{E}[\mathcal{L}_{(t+1)\bar{E}+\frac{1}{2}}] \leq \mathbb{E}[\mathcal{L}_{(t+1)\bar{E}}] + w_2 L_2 \eta \bar{E} G. \quad (14)$$

Its proof is also shown in the appendix. Lastly, we can obtain the convergence rate as follows:

Theorem 1. (Non-convex convergence rate) Let **Assumption 1, 2, 3, and 4** hold and assume $\mathcal{L}^*$ is the local optimal. For any end device training, its convergence rate is bound as

$$\begin{aligned} \frac{1}{T\bar{E}} \sum_{t=0}^{T-1} \sum_{\bar{e}=0}^{\bar{E}-1} \|\nabla \mathcal{L}_{t\bar{E}+\bar{e}}\|_2^2 &\leq \frac{\mathbb{E}[\mathcal{L}_0 - \mathcal{L}^*]}{\eta \bar{E} T} \\ &\quad + \frac{L_1 \eta G^2}{2} + w_2 L_2 G. \end{aligned} \quad (15)$$

The proof is shown in the appendix. It should be noted that there is a non-negligible error term $w_2 L_2 \eta \bar{E} G$ in Eq. (15). It appears from the lower bound of the norm of the stochastic gradient (Assumption 3). As the number of global communication rounds T increases, the gradient eventually becomes zero or very small. Therefore, PAECL can converge to a neighborhood of the stationary point.

6 Experiments

6.1 Simulation Experiment

Datasets and scenario. We conduct experiments on four image classification datasets (MNIST, CIFAR-10, CIFAR-100, and Tiny-ImageNet) and an audio-recognition dataset (Google Speech Commands). If not specified, we simulate a large network scenario

including 4 edge servers and 80 end devices and randomly set their affiliations. We randomly distribute the original training and test sets to all end devices as local data. To simulate non-independent and identically distributed data in reality, we mix the training and test sets as the total dataset. All edge servers only have a corresponding test set of the original dataset without training datasets. We set the Dirichlet distribution parameter α to 0.5 and 0.1 to simulate the low and high heterogeneity scenarios, respectively.

Comparison approaches. We extend four representative methods to the above network scenario since they only work with one edge server and cannot be directly compared. These methods cover homogeneous and heterogeneous edge-end model training. **EAvG**: It collaborates model training among cloud, edge, and end by transferring gradients in a layered manner [21]. **EProto**: It is adapted from FedProto to multiple edge servers and end devices scenarios [31]. **EGkt**: It trains different scale CNN models by transferring logits [9]. **FedAgg**: It trains edge-end heterogeneous models by transferring bridge samples [38].

Evaluation metrics. Our metrics include the average accuracy of end models and the highest test accuracy of edge models.

Implementation detail. We refer to parameter settings in [31, 40, 47]. For the MNIST dataset, the end model is a two-layer CNN, and the edge model is ResNet-10. In CIFAR-10, CIFAR-100, and Tiny-ImageNet datasets, the end model is ResNet-8, and the edge model is ResNet-34. For the Google Speech Command dataset, we train VGG-8 on end devices and VGG-16 on edge servers. In every edge server, the pre-trained generative model is StyleGAN [12]. We train all the models for 100 iterations with batch size 10, and Adam optimizer with 0.005 initial learning rate, and the learning rate is multiplied by 0.9 every 20 iterations. To avoid accidental results, all experiments are run three times and averaged.

Ablation experiments. With the former implementation detail, we conduct ablation experiments on the CIFAR-10 dataset ($\alpha = 0.1$). For PAECL, we remove local contrast (GA+EA), global alignment (LC+EA), and edge augmentation (LC+GA). Then we obtain the performance of end and edge models on the CIFAR-10 dataset.

Comparison results. In the high heterogeneity scenario ($\alpha = 0.1$), our PAECL exhibits excellent performance in both edge models and end models. Specifically, for the end models, Fig. 3(a) shows in the MNIST dataset, PAECL achieves the average test accuracy of $99.1 \pm 0.12\%$, outperforming all the other methods. Fig. 3(b) shows that in the CIFAR-10 dataset, PAECL outperforms EAvG, EProto, EGkt, and FedAgg at average test accuracy by 45.21%, 2.03%, 3.35%, and 32.14%. Fig. 3(c) illustrates that PAECL yields superior performance in the CIFAR-100 dataset, surpassing EAvG, EProto, EGkt, and FedAgg by 37.82%, 3.21%, 9.43%, and 36.21% in terms of average test accuracy. Similarly, in the Tiny-Imagenet dataset, Fig. 3(d) shows that PAECL outperforms EAvG, EProto, EGkt, and FedAgg by 17.16%, 4.14%, 15.14%, and 22.94% in terms of average test accuracy. As illustrated in Fig. 5(a), PAECL effectively facilitates heterogeneous edge model training by collaboration with end models, demonstrating better performance of at least 13.12%, 17.96%, 8.49%, and 6.75% in the highest accuracy compared to FedAgg and EGkt across the four datasets, respectively. On the audio-recognition dataset, as shown in Fig. 7, PAECL also achieves higher accuracy compared with other methods.

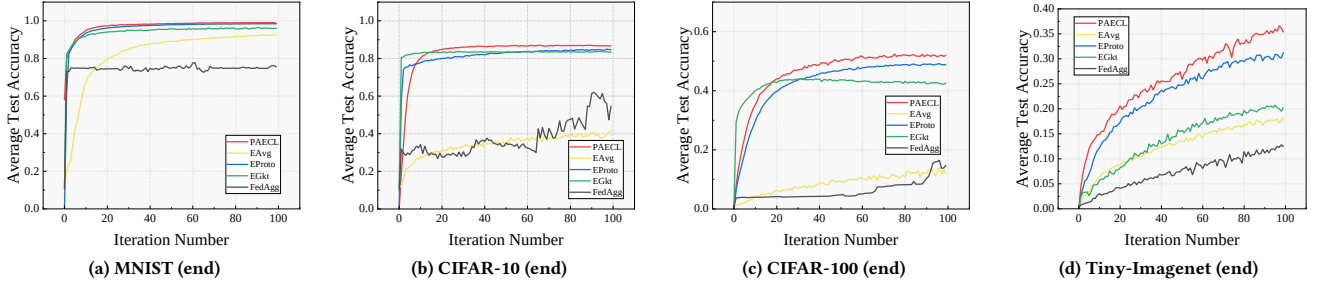


Figure 3: Average accuracy of end model on four image classification datasets ($\alpha = 0.1$).

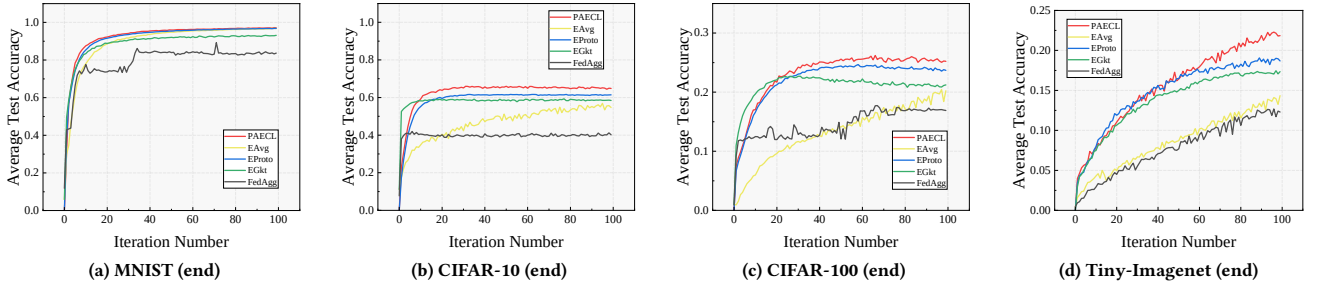


Figure 4: Average accuracy of end model on four image classification datasets ($\alpha = 0.5$).

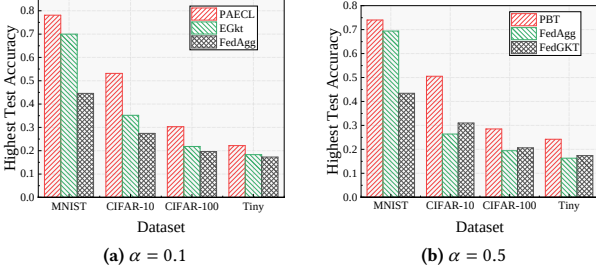


Figure 5: Accuracy of edge model on different datasets.

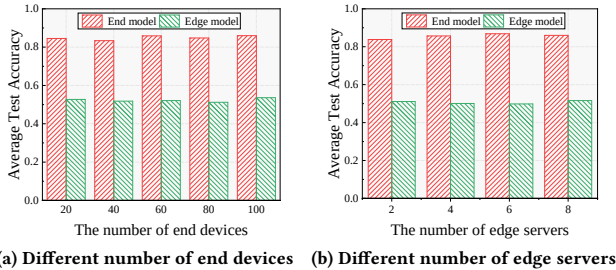


Figure 6: Average test accuracy with different numbers of end devices and edge servers.

In the low heterogeneity scenario ($\alpha = 0.5$), although the accuracy of models trained by all methods decreases, Figs. 4 and 7 show

that PAECL still has the best performance among all the methods in different datasets. For example, in the CIFAR-100 dataset, PAECL outperforms EAvg, EProto, EGkt, and FedAgg at average test accuracy by 4.97%, 1.63%, 3.98%, and 8.32%. For the edge model, in Fig. 5, PAECL surpasses other methods on the performance of the edge model 5.61%, 19.51%, 7.86%, and 6.80% respectively on four datasets by augmenting the prototype.

In addition, we also evaluate the performance of PAECL under different numbers of end devices and different numbers of edge servers. We use the CIFAR-10 dataset and randomly divide it for different end devices. As shown in Fig. 6(a), when the number of edge servers is fixed at 4, as the number of end devices increases from 20 to 100, the end model accuracy and edge model accuracy trained by PAECL remain stable. Similarly, we fix the number of end devices to 40 and increase the number of edge servers from 2 to 8. Fig. 6(b) shows that the average accuracy of the edge models and end models is stable around 83.72% and 49.83%, respectively. Therefore, PAECL performs stably even in training with different numbers of end devices and edge servers.

Last, ablation experiments are shown in Fig. 8. Our proposed PAECL method incorporates LC, GA, and EA, where LC and GA are employed to enhance end models, and EA is designed to train edge models on servers. We analyse results from three perspectives: (1) LC: PAECL obtains an accuracy drop of 4.54% and 3.82%. This is because the loss of local contrast leads to a decrease in prototype representation capabilities, which also affects the edge model accuracy. (2) GA: A similar decrease is also reflected in LC+EA, and global alignment causes the model accuracy to decrease by at least 5.03%. (3) EA: Losing edge enhancement has no impact on end

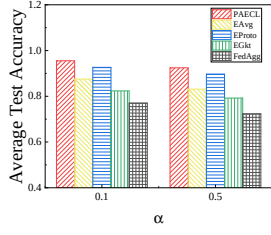


Figure 7: Average test accuracy on audio-recognition dataset. (PAECL contains LC+GA+SA).

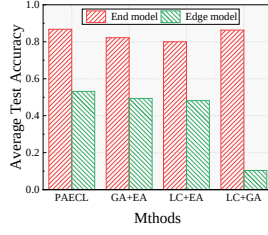


Figure 8: Ablation experiments (PAECL contains LC+GA+SA).

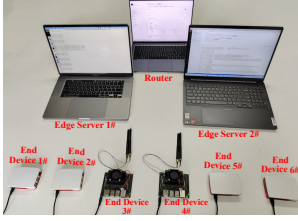


Figure 9: Field experimental scenario.

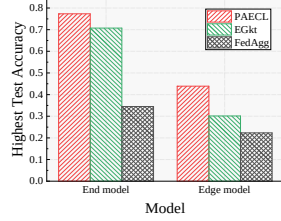


Figure 10: The highest accuracies of edge-end models.

Methods	Time	Burden	End Acc.	Edge Acc.
PAECL	138.56s	0.11MB	76.62±0.96%	43.22±1.02%
EAvg	72.47s	3.05MB	35.61±0.36%	-
EProto	116.24s	0.11MB	70.09±0.73%	-
EGkt	155.34s	32.86MB	69.28±1.75%	29.45±0.42%
FedAgg	365.32s	59.86MB	33.47±1.33%	21.54±2.61%

Table 2: Average training clock time, average transmission burden, average accuracy of end and edge models in the CIFAR-10 dataset. "-" means the method cannot support heterogeneous edge model training.

model accuracy, but it causes the edge model to degenerate into a randomly initialized state.

6.2 Field Experiment

Datasets and scenario. The simulation experiments in the previous section verify the performance of PAECL. To evaluate it further, we construct an edge-end experimental scenario with real heterogeneous devices. As shown in Fig. 9, an edge-end collaborative system consists of 2 PCs as edge servers and 6 end devices including 4 Raspberry Pi 4 Model B with 8GB RAM and 2 NVIDIA Jetson Nano. End devices 1#, 2#, and 3# collaborate with Edge server 1#, and Edge devices 4#, 5#, and 6# collaborate with edge server 2#. Edge server 2# is selected as the aggregated edge server. Besides, a PC acts as a router to connect two edge servers for transmitting edge prototypes and synchronizing the global prototype.

Evaluation metrics. We ran 50 communication rounds to evaluate the performance of all methods. We calculate the average training time and average transmission burden of these 50 rounds. The average training time is defined as the total time required for training divided by the number of rounds. The average communication

overhead is defined as the total amount of knowledge transmitted for knowledge sharing divided by the number of rounds. Besides, we use the highest accuracy of edge models and the average accuracy of end models among all the devices to evaluate different training methods.

Implementation detail. All end devices are equipped with 3000 training images and 1000 testing images from the CIFAR-10 dataset. The edge model is ResNet-8, and the end model is a two-layer CNN.

Comparison results. As shown in Table 2, EAvg and EProto cannot train heterogeneous edge models. Therefore, their edge model accuracy is presented as empty and the average consumed training time is lower than PAECL. In sharp contrast, PAECL not only supports heterogeneous edge model training but also achieves the highest average end model accuracy among all the comparison approaches. After being augmented by end devices and edge servers, the augmented prototype is the only knowledge transmitted by PAECL. Therefore, PAECL achieves the lowest transmission cost of 0.11M. PAECL only operates the prototype, a lightweight vector for training and augmentation, so its average training time per round on the end device is 138.56s and lower than other methods. The end model obtained after PAECL training also has higher accuracy than EAvg and EProto, 41.01% and 6.53% respectively.

Although EGkt and FedAgg methods support heterogeneous model training, PAECL achieves the lowest clock time. It saves at least 297 times the transmission burden and 11.2% of training clock time, achieving higher accuracies of the edge model and end model. For the edge model, PAECL enables the edge model to achieve the highest accuracy of 43.22%, which is superior to EGkt and FedAgg by 13.77% and 21.68%. Fig. 10 shows the highest accuracies of edge models and end models of all the methods proposed for edge-end heterogeneous training. Under the collaborative training of 6 end devices and 2 edge servers, the highest accuracies of edge models and end models trained by PAECL are 77.34% and 43.89%, which are at least 6.61% and 13.77% higher than EGkt and FedAgg.

7 Conclusion

This paper proposes PAECL, an edge-end heterogeneous collaborative learning method by transmitting prototypes. It innovatively augments local prototypes by prototype contrast and alignment on end devices, and augments the global prototype by mixing it with noise on edge servers. The augmented prototype not only guides end device training but also generates data for edge servers to train larger models. Through simulation and field experiments, we prove that PAECL achieves higher accuracy and lower training time than other methods with negligible transmission burden.

Acknowledgments

This work was supported in part by the National Key R&D Program of China under Grant No. 2023YFB4502400, in part by the Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China under Grant No. JYB2025XDXM901, in part by the National Natural Science Foundation of China under Grant No. 62272223, U22A2031, in part by the Collaborative Innovation Center of Novel Software Technology and Industrialization, Nanjing University, and in part by the Jiangsu High-level Innovation and Entrepreneurship (Shuangchuang) Program. Haipeng Dai is the corresponding author.

References

- [1] Zhao Bai, Mingyue Wang, Fangda Guo, Yu Guo, Chengjun Cai, Rongfang Bie, and Xiaohua Jia. 2024. SecMdp: Towards Privacy-Preserving Multimodal Deep Learning in End-Edge-Cloud. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, Utrecht, 1659–1670.
- [2] Chen Chen, Chenyu Wang, Bin Liu, Ci He, Li Cong, and Shaohua Wan. 2023. Edge intelligence empowered vehicle detection and image segmentation for autonomous vehicles. *IEEE Transactions on Intelligent Transportation Systems* 24, 11 (2023), 13023–13034.
- [3] Zhipeng Cheng, Xiaoyu Xia, Minghui Liwang, Xuwei Fan, Yanglong Sun, Xianbin Wang, and Lianfen Huang. 2023. Cheese: Distributed clustering-based hybrid federated split learning over edge networks. *IEEE Transactions on Parallel and Distributed Systems* 34, 12 (2023), 3174–3191.
- [4] Yutong Dai, Zeyuan Chen, Junnan Li, Shelby Heinecke, Lichao Sun, and Ran Xu. 2023. Tackling data heterogeneity in federated learning with class prototypes. In *Proceedings of the AAAI Conference on Artificial Intelligence*. AAAI Press, Washington, DC, 7314–7322.
- [5] Jessica Deuschel, Daniel Firmbach, Carol I Geppert, Markus Eckstein, Arndt Hartmann, Volker Bruns, Petr Kuritcyn, Jakob Dexl, David Hartmann, Dominik Perrin, et al. 2021. Multi-prototype few-shot learning in histopathology. In *Proceedings of the IEEE/CVF international conference on computer vision*. IEEE, Montreal, 620–628.
- [6] Junyu Gao, Xinhong Ma, and Changsheng Xu. 2024. Learning transferable conceptual prototypes for interpretable unsupervised domain adaptation. *IEEE Transactions on Image Processing* 33, 1 (2024), 5284 – 5297.
- [7] Bin Gu, An Xu, Zhouyuan Huo, Cheng Deng, and Heng Huang. 2021. Privacy-preserving asynchronous vertical federated learning algorithms for multiparty collaborative learning. *IEEE transactions on neural networks and learning systems* 33, 11 (2021), 6103–6115.
- [8] Yeting Guo, Fang Liu, Tongqing Zhou, Zhipeng Cai, and Nong Xiao. 2023. Privacy vs. Efficiency: Achieving Both Through Adaptive Hierarchical Federated Learning. *IEEE Transactions on Parallel and Distributed Systems* 34, 4 (2023), 1331–1342.
- [9] Chaoyang He, Murali Annavaram, and Salman Avestimehr. 2020. Group knowledge transfer: Federated learning of large cnns at the edge. In *Proceedings of the Advances in Neural Information Processing Systems*. Curran Associates Inc., virtual, 14068–14080.
- [10] Sujin Hou, Enze Yu, Fang Mei, Yuben Qu, Haihan Zhang, and Haipeng Dai. 2025. Prototype-Based Semi-Asynchronous Edge-End Collaborative Learning with Client Clustering. In *2025 IEEE 50th Conference on Local Computer Networks (LCN)*. IEEE, Sydney, 1–8.
- [11] Wenke Huang, Mang Ye, Zekun Shi, He Li, and Bo Du. 2023. Rethinking federated learning with domain shift: A prototype view. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. IEEE, Vancouver, 16312–16322.
- [12] Tero Karras, Miika Aittala, Janne Hellsten, Samuli Laine, Jaakko Lehtinen, and Timo Aila. 2020. Training generative adversarial networks with limited data. In *Proceedings of the Advances in Neural Information Processing Systems*. Curran Associates Inc., virtual, 12104–12114.
- [13] Prannay Khosla, Piotr Teterwak, Chen Wang, Aaron Sarna, Yonglong Tian, Phillip Isola, Aaron Maschinot, Ce Liu, and Dilip Krishnan. 2020. Supervised contrastive learning. In *Proceedings of the Advances in Neural Information Processing Systems*. Curran Associates Inc., virtual, 18661–18673.
- [14] Yuxin Kong, Peng Yang, and Yan Cheng. 2023. Edge-assisted on-device model update for video analytics in adverse environments. In *Proceedings of the ACM International Conference on Multimedia*. ACM, Ottawa, 9051–9060.
- [15] Ang Li, Jingwei Sun, Pengcheng Li, Yu Pu, Hai Li, and Yiran Chen. 2021. Hermes: an efficient federated learning framework for heterogeneous mobile clients. In *Proceedings of the Annual International Conference on Mobile Computing and Networking*. ACM, New Orleans, 420–437.
- [16] Xiang Li, Kaixuan Huang, Wenhao Yang, Shusen Wang, and Zhihua Zhang. 2020. On the Convergence of FedAvg on Non-IID Data. In *International Conference on Learning Representations*. Openreview.net, Addis Ababa, 1–26.
- [17] Zexi Li, Qunwei Li, Yi Zhou, Wenliang Zhong, Guannan Zhang, and Chao Wu. 2023. Edge-cloud collaborative learning with federated and centralized features. In *Proceedings of the International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, Taipei, 1949–1953.
- [18] Yunming Liao, Yang Xu, Hongli Xu, Lun Wang, Zhiwei Yao, and Chunming Qiao. 2024. Mergesfl: Split federated learning with feature merging and batch size regulation. In *IEEE International Conference on Data Engineering*. IEEE, Utrecht, 2054–2067.
- [19] Yunming Liao, Yang Xu, Hongli Xu, Zhiwei Yao, Liusheng Huang, and Chunming Qiao. 2024. Parallelsfl: A novel split federated learning framework tackling heterogeneity issues. In *Proceedings of the Annual International Conference on Mobile Computing and Networking*. ACM, Washington, 845–860.
- [20] Jinlu Liu, Liang Song, and Yongqiang Qin. 2020. Prototype rectification for few-shot learning. In *European conference on computer vision*. Springer, Glasgow, 741–756.
- [21] Lumin Liu, Jun Zhang, SH Song, and Khaled B Letaief. 2020. Client-edge-cloud hierarchical federated learning. In *Proceedings of the IEEE international conference on communications*. IEEE, Dublin, 1–6.
- [22] Linfeng Liu, Yaoze Zhou, and Jia Xu. 2024. A Cloud-Edge-End Collaboration Framework for Cruising Route Recommendation of Vacant Taxis. *IEEE Transactions on Mobile Computing* 23, 5 (2024), 4678–4693.
- [23] Jing Long, Guanhua Ye, Tong Chen, Yang Wang, Meng Wang, and Hongzhi Yin. 2024. Diffusion-Based Cloud-Edge-Device Collaborative Learning for Next POI Recommendations. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. ACM, Barcelona, 2026–2036.
- [24] Jianghu Lu, Shikun Li, Kexin Bao, Pengju Wang, Zhenxing Qian, and Shiming Ge. 2023. Federated Learning with Label-Masking Distillation. In *Proceedings of the ACM International Conference on Multimedia*. ACM, Ottawa, 222–232.
- [25] Feng Lyu, Cheng Tang, Yongheng Deng, Tong Liu, Yongmin Zhang, and Yaoyue Zhang. 2023. A prototype-based knowledge distillation framework for heterogeneous federated learning. In *IEEE International Conference on Distributed Computing Systems*. IEEE, Hong Kong, 1–11.
- [26] Yingwei Pan, Ting Yao, Yehao Li, Yu Wang, Chong-Wah Ngo, and Tao Mei. 2019. Transferrable prototypical networks for unsupervised domain adaptation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. IEEE, Long Beach, 2239–2247.
- [27] Yu Qiao, Md Shirajum Munir, Apurba Adhikary, Huy Q Le, Avi Deb Raha, Chaoning Zhang, and Choong Seon Hong. 2023. Mp-fedcl: Multiprototype federated contrastive learning for edge intelligence. *IEEE Internet of Things Journal* 11, 5 (2023), 8604–8623.
- [28] Ariadna Quattoni, Michael Collins, and Trevor Darrell. 2008. Transfer learning for image classification with sparse prototype representations. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, Anchorage, 1–8.
- [29] Jake Snell, Kevin Swersky, and Richard Zemel. 2017. Prototypical networks for few-shot learning. In *Proceedings of the Advances in Neural Information Processing Systems*. Curran Associates Inc., Long Beach, 1–11.
- [30] Shangquan Sun, Wenqi Ren, Jingzhi Li, Rui Wang, and Xiaochun Cao. 2024. Logit standardization in knowledge distillation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. IEEE, Seattle, 15731–15740.
- [31] Yue Tan, Guodong Long, Lu Liu, Tianyi Zhou, Qinghua Lu, Jing Jiang, and Chengqi Zhang. 2022. Fedproto: Federated prototype learning across heterogeneous clients. In *Proceedings of the AAAI Conference on Artificial Intelligence*. AAAI Press, Virtual, 8432–8440.
- [32] Trung-Kien Tran, Ha-Phuong Tran, Thi-Lan Le, and Thanh-Hai Tran. 2024. FedNTProto: A Prototype-Based Approach for Personalized Federated Learning. In *International Conference on Multimedia Analysis and Pattern Recognition*. IEEE, Da Nang, 1–6.
- [33] Gunet Kaur Walia, Mohit Kumar, and Sukhpal Singh Gill. 2023. AI-empowered fog/edge resource management for IoT applications: A comprehensive review, research challenges and future perspectives. *IEEE Communications Surveys & Tutorials* 26, 1 (2023), 619–669.
- [34] Guanqun Wang, Jiaming Liu, Chenxuan Li, Yuan Zhang, Junpeng Ma, Xinyu Wei, Kevin Zhang, Maurice Chong, Renrui Zhang, Yijiang Liu, et al. 2024. Cloud-Device Collaborative Learning for Multimodal Large Language Models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. IEEE, Seattle, 12646–12655.
- [35] Kaibin Wang, Qiang He, Feifei Chen, Chunyang Chen, Faliang Huang, Hai Jin, and Yun Yang. 2023. Flexifed: Personalized federated learning for edge clients with heterogeneous model architectures. In *Proceedings of the ACM Web Conference*. IEEE, Austin, 2979–2990.
- [36] Song Wang and Xinyu Zhang. 2022. NeuroMessenger: Towards error tolerant distributed machine learning over edge networks. In *IEEE INFOCOM 2022-IEEE Conference on Computer Communications*. IEEE, London, 2058–2067.
- [37] Jiajun Wu, Fan Dong, Henry Leung, Zhuangdi Zhu, Jiayu Zhou, and Steve Drew. 2024. Topology-aware federated learning in edge computing: A comprehensive survey. *Comput. Surveys* 56, 10 (2024), 1–41.
- [38] Zhiyuan Wu, Sheng Sun, Yuwei Wang, Min Liu, Bo Gao, Quyang Pan, Tianliu He, and Xuefeng Jiang. 2024. Agglomerative federated learning: Empowering larger model training via end-edge-cloud collaboration. In *Proceedings of the IEEE Conference on Computer Communications*. IEEE, Vancouver, 131–140.
- [39] Zhiyuan Wu, Sheng Sun, Yuwei Wang, Min Liu, Quyang Pan, Xuefeng Jiang, and Bo Gao. 2023. Fedict: Federated multi-task distillation for multi-access edge computing. *IEEE Transactions on Parallel and Distributed Systems* 35, 6 (2023), 1107–1121.
- [40] Yi Xu, Ying Li, Haoyu Luo, Xiaoliang Fan, and Xiao Liu. 2024. FBLG: A Local Graph Based Approach for Handling Dual Skewed Non-IID Data in Federated Learning. In *Proceedings of the International Joint Conference on Artificial Intelligence, IJCAI-24*. ijcai.org, Jeju, 5289–5297.
- [41] Yang Xu, Lun Wang, Hongli Xu, Jianchun Liu, Zhiyuan Wang, and Liusheng Huang. 2023. Enhancing federated learning with server-side unlabeled data by adaptive client and data selection. *IEEE Transactions on Mobile Computing* 23, 4

- (2023), 2813–2831.
- [42] Helin Yang, Kwok-Yan Lam, Liang Xiao, Zehui Xiong, Hao Hu, Dusit Niyato, and H Vincent Poor. 2022. Lead federated neuromorphic learning for wireless edge artificial intelligence. *Nature communications* 13, 1 (2022), 4269.
- [43] Jiangchao Yao, Shengyu Zhang, Yang Yao, Feng Wang, Jianxin Ma, Jianwei Zhang, Yunfei Chu, Luo Ji, Kunyang Jia, Tao Shen, et al. 2022. Edge-cloud polarization and collaboration: A comprehensive survey for ai. *IEEE Transactions on Knowledge and Data Engineering* 35, 7 (2022), 6866–6886.
- [44] Zhiwei Yao, Jianchun Liu, Hongli Xu, Lun Wang, Chen Qian, and Yunming Liao. 2024. Ferrari: A personalized federated learning framework for heterogeneous edge clients. *IEEE Transactions on Mobile Computing* 23, 10 (2024), 10031–10045.
- [45] Mang Ye, Xiuwen Fang, Bo Du, Pong C. Yuen, and Dacheng Tao. 2023. Heterogeneous Federated Learning: State-of-the-art and Research Challenges. *ACM Computing Survey* 56, 3, Article 79 (Oct. 2023), 44 pages.
- [46] Rui Ye, Zhenyang Ni, Fangzhao Wu, Siheng Chen, and Yanfeng Wang. 2023. Personalized federated learning with inferred collaboration graphs. In *Proceedings of the International Conference on Machine Learning*. PMLR, Honolulu, 39801–39817.
- [47] Rui Ye, Minghai Xu, Jianyu Wang, Chenxin Xu, Siheng Chen, and Yanfeng Wang. 2023. Feddisco: Federated learning with discrepancy-aware collaboration. In *Proceedings of the International Conference on Machine Learning*. PMLR, Honolulu, 39879–39902.
- [48] Enze Yu, Haipeng Dai, Haihan Zhang, Zhenzhe Zheng, Jun Zhao, and Guihai Chen. 2025. Prototype-Based Collaborative Learning in UAV-Assisted Edge Computing Networks. *Software: Practice and Experience* 55, 5 (2025), 829–846.
- [49] Chen Zhang, Yu Xie, Tingbin Chen, Wenjie Mao, and Bin Yu. 2024. Prototype Similarity Distillation for Communication-Efficient Federated Unsupervised Representation Learning. *IEEE Transactions on Knowledge and Data Engineering* 36, 11 (2024), 6865–6876. doi:10.1109/TKDE.2024.3386712
- [50] Jianqing Zhang, Yang Liu, Yang Hua, and Jian Cao. 2024. Fedtgp: Trainable global prototypes with adaptive-margin-enhanced contrastive learning for data and model heterogeneity in federated learning. In *Proceedings of the AAAI conference on Artificial Intelligence*. AAAI Press, Vancouver, 16768–16776.

A Proof of Lemma 1

Proof. For any end device, $\omega_{t\bar{E}+1} = \omega_{t\bar{E}} - \eta g_{t\bar{E}}$ holds, then we have

$$\begin{aligned} \mathcal{L}_{t\bar{E}+1} &\stackrel{(a)}{\leq} \mathcal{L}_{t\bar{E}} + \langle \nabla \mathcal{L}_{t\bar{E}}, (\omega_{t\bar{E}+1} - \omega_{t\bar{E}}) \rangle + \frac{L_1}{2} \|\omega_{t\bar{E}+1} - \omega_{t\bar{E}}\|_2^2 \\ &= \mathcal{L}_{t\bar{E}} - \eta \langle \nabla \mathcal{L}_{t\bar{E}}, g_{t\bar{E}} \rangle + \frac{L_1}{2} \|\eta g_{t\bar{E}}\|_2^2. \end{aligned} \quad (16)$$

Taking the expectation of both sides for the above equations, then we have

$$\begin{aligned} \mathbb{E}[\mathcal{L}_{t\bar{E}+1}] &\leq \mathbb{E}[\mathcal{L}_{t\bar{E}}] - \eta \mathbb{E}[\langle \nabla \mathcal{L}_{t\bar{E}}, g_{t\bar{E}} \rangle] + \frac{L_1 \eta^2}{2} \mathbb{E}[\|g_{t\bar{E}}\|_2^2] \\ &\stackrel{(b)}{=} \mathbb{E}[\mathcal{L}_{t\bar{E}}] - \eta \|\nabla \mathcal{L}_{t\bar{E}}\|_2^2 + \frac{L_1 \eta^2}{2} \mathbb{E}[\|g_{t\bar{E}}\|_2^2] \\ &\stackrel{(c)}{\leq} \mathbb{E}[\mathcal{L}_{t\bar{E}}] - \eta \|\nabla \mathcal{L}_{t\bar{E}}\|_2^2 + \frac{L_1 \eta^2}{2} G^2, \end{aligned} \quad (17)$$

where (a) follows from L_1 -smooth bound, (b) follows from **Assumption 2**, and (c) follows **Assumption 3**. Then, rearranging and summing $\bar{E}$ steps, we prove the above lemma. ■

B Proof of Lemma 2

Proof. We add a zero term to $\mathcal{L}_{(t+1)\bar{E}+\frac{1}{2}}$, then we have

$$\begin{aligned} \mathcal{L}_{(t+1)\bar{E}+\frac{1}{2}} &= \mathcal{L}_{(t+1)\bar{E}} + \mathcal{L}_{(t+1)\bar{E}+\frac{1}{2}} - \mathcal{L}_{(t+1)\bar{E}} \\ &\stackrel{(a)}{\leq} \mathcal{L}_{(t+1)\bar{E}} + w_2 \|\bar{p}_{(t+1)\bar{E}} - P_{g,(t+2)\bar{E}}\|_2 - w_2 \|\bar{p}_{(t+1)\bar{E}} - P_{g,(t+1)\bar{E}}\|_2 \\ &\stackrel{(b)}{=} \mathcal{L}_{(t+1)\bar{E}} + w_2 \left\| \sum_{i=1}^N q_i \bar{p}_{i,(t+1)\bar{E}} - \sum_{i=1}^N q_i \bar{p}_{i,t\bar{E}} \right\|_2 \\ &= \mathcal{L}_{(t+1)\bar{E}} + w_2 L_2 \sum_{i=1}^N q_i \|\bar{p}_{i,(t+1)\bar{E}} - \bar{p}_{i,t\bar{E}}\|_2 \\ &\stackrel{(c)}{\leq} \mathcal{L}_{(t+1)\bar{E}} + w_2 L_2 \sum_{i=1}^N q_i \|\omega_{i,(t+1)\bar{E}} - \omega_{i,t\bar{E}}\|_2 \\ &\stackrel{(d)}{\leq} \mathcal{L}_{(t+1)\bar{E}} + w_2 L_2 \eta \sum_{i=1}^N q_i \sum_{\bar{e}=0}^{\bar{E}-1} \|g_{i,t\bar{E}+\bar{e}}\|_2, \end{aligned} \quad (18)$$

where $q_i = \frac{|D_i|}{\sum_{i=1}^N |D_i|}$, and it represents weights. (a) follows the definition of local loss function for all end devices; (b) follows the definitions of the global, edge, and end prototypes; (c) follows from L_2 -Lipschitz smooth bound; and (d) follows the definition of gradient. Then we take expectations on both sides of the above equation, and we have

$$\begin{aligned} \mathbb{E}[\mathcal{L}_{(t+1)\bar{E}+\frac{1}{2}}] &\leq \mathbb{E}[\mathcal{L}_{(t+1)\bar{E}}] + w_2 L_2 \eta \sum_{\bar{e}=0}^{\bar{E}-1} \mathbb{E}[\|g_{t\bar{E}+\bar{e}}\|_2] \\ &\stackrel{(a)}{\leq} \mathbb{E}[\mathcal{L}_{(t+1)\bar{E}}] + w_2 L_2 \eta \bar{E} G, \end{aligned} \quad (19)$$

and (a) follows **Assumption 3**. ■

C Proof of Theorem 1

Proof. Substituting **Lemma 1** to **Lemma 2**, we have

$$\begin{aligned} \eta \sum_{\bar{e}=0}^{\bar{E}-1} \|\nabla \mathcal{L}_{t\bar{E}+\bar{e}}\|_2^2 &\leq \mathbb{E}[\mathcal{L}_{t\bar{E}} - \mathcal{L}_{t(\bar{E}+1)+\frac{1}{2}}] + \\ &\quad \frac{L_1 \eta^2 \bar{E} G^2}{2} + w_2 L_2 \eta \bar{E} G, \end{aligned} \quad (20)$$

rearranging and summing t from 0 to $T-1$ (T rounds), then we prove the convergence rate in Eq. (15). ■